

Investigating and Implementing LibreRouter and LibreMesh: Virtualized and Raspberry Pi-Based Mesh Networking

Bastas, Dimitrios (SUPREMER LEFTIEUENT COMMANDER)
Kamilos, Dimitrios (SUPREMEISH RIGHTIEUNENT COMMANDER)
Robert Leyba (LT)
Rachel Goldstein (LT)
Margaret Moore (LTJG)
EC3730 Final Project

Abstract—This project investigates LibreRouter and LibreMesh, open-source community networking technologies built on top of OpenWrt. The project explores the architecture, implementation, vulnerabilities, and deployment challenges associated with decentralized mesh networking systems.

A virtualized OpenWrt environment was developed using VMware Workstation Pro to simulate a four-node routed mesh backbone topology. The environment established reproducible experimentation capabilities for distributed routing analysis, virtualization behavior, and future LibreMesh deployment.

The project will later extend the deployment onto Raspberry Pi hardware platforms for physical mesh networking experimentation and wireless performance evaluation.

I. INTRODUCTION

Community mesh networks provide decentralized communication infrastructure capable of operating independently of centralized Internet Service Providers (ISPs). LibreRouter and LibreMesh were developed to support resilient, community-owned wireless infrastructure using open-source software and commodity hardware.

This project investigates the deployment and operation of LibreMesh-based networking systems using both virtualized and physical implementations. The primary goals include evaluating routing behavior, deployment complexity, security considerations, network resilience, and practical limitations of community mesh networking technologies.

In addition to traditional research analysis, this report maintains a reproducible engineering notebook style documenting implementation steps, troubleshooting observations, operational anomalies, and experimental outcomes throughout the project lifecycle.

II. BACKGROUND RESEARCH

A. OpenWrt

OpenWrt is an open-source Linux-based operating system designed primarily for embedded networking devices such as routers and access points. Unlike traditional vendor firmware, OpenWrt provides a writable filesystem, package management system, and modular architecture allowing users to customize routing, firewall, wireless, and network management functionality.

The OpenWrt ecosystem supports x86 virtualized environments in addition to physical embedded hardware, making it suitable for experimental mesh networking research.

B. LibreMesh

LibreMesh is a framework built on top of OpenWrt that simplifies deployment of decentralized mesh networking infrastructure. The platform automates many aspects of wireless mesh configuration while supporting routing protocols such as BATMAN-adv and BMX7.

LibreMesh was designed to support community-operated networks where nodes dynamically discover neighboring devices and establish resilient communication paths without centralized infrastructure.

C. LibreRouter

LibreRouter is an open-hardware networking platform developed specifically for community mesh deployments. The platform combines OpenWrt-based firmware with wireless hardware optimized for long-range and community-managed networking applications.

III. LITERATURE REVIEW AND SECURITY ANALYSIS

A. Identified Weaknesses and Vulnerabilities

Several security concerns and operational limitations were identified during preliminary literature review:

- Firmware update vulnerabilities in OpenWrt package management systems
- Mesh routing attacks including route poisoning and malicious node participation
- Wireless sniffing and unauthorized network access
- Resource exhaustion and denial-of-service attacks
- Complexity of maintaining decentralized routing infrastructure

IV. VIRTUALIZED OPENWRT DEPLOYMENT

A. Virtualization Environment

VMware Workstation Pro 17 was selected as the virtualization platform for the initial implementation phase. Virtualization provided rapid deployment, rollback capability through

snapshots, reproducible experimentation, and simplified troubleshooting prior to deployment on physical Raspberry Pi hardware.

B. OpenWrt Image Preparation

The OpenWrt x86_64 ext4 combined image was downloaded from the official OpenWrt release repository. Since VMware does not natively boot raw IMG files efficiently, the OpenWrt disk image was converted into VMware VMDK format prior to deployment.

C. Virtual Machine Configuration

A custom virtual machine configuration was selected in VMware Workstation Pro to allow manual control over networking interfaces and storage configuration.

TABLE I
INITIAL OPENWRT VM CONFIGURATION

Setting	Value
Guest OS	Other Linux 5.x 64-bit
vCPUs	1
Memory	256 MB
Primary Disk	OpenWrt VMDK
Network Adapter 1	NAT
Network Adapter 2	Host-only

D. Network Interface Design

Two virtual network adapters were configured for each virtual router node:

- NAT adapter for WAN connectivity and package installation
- Host-only adapter for internal mesh backbone communication

This architecture allowed the virtual environment to simulate isolated routed mesh connectivity while preserving Internet access for package installation and updates.

E. Initial Boot Validation

The OpenWrt virtual machine booted successfully after the VMDK disk image was attached to the VMware virtual SCSI controller.

During startup, OpenWrt initialized both virtual Ethernet interfaces successfully:

```
eth0 NIC Link is Up
eth1 NIC Link is Up
```

The successful initialization confirmed that VMware virtual networking devices were correctly recognized by the OpenWrt kernel.



Fig. 1. Successful OpenWrt boot and interface initialization.

V. NETWORK TROUBLESHOOTING AND RECOVERY

A. Initial Connectivity Failure

Initial deployment attempts failed to provide Internet connectivity.

Observed symptoms included:

- Missing default routes
- Failed DNS resolution
- Inability to ping external IP addresses
- DHCP inconsistencies

Example failure:

```
ping -c 4 8.8.8.8
Network unreachable
```

Investigation revealed that VMware host-only networking was unintentionally acting as the WAN DHCP source. Multiple adapter configurations caused interface ambiguity and routing instability.

Observed VMware virtual networks included:

- VMnet1: 192.168.61.0/24
- VMnet8: 192.168.111.0/24

B. Controlled Simplification Strategy

To isolate the networking issue, the second adapter was temporarily removed and the deployment simplified to a single NAT interface.

After simplification, the OpenWrt node successfully received:

- DHCP assignment
- Default gateway
- External DNS access

Working configuration:

```
default via 192.168.111.2 dev eth0
```

Successful validation tests included:

```
ping -c 4 8.8.8.8
ping -c 4 google.com
```

All tests completed successfully with zero packet loss.

C. Package Management Failure Analysis

Package management operations initially failed during APK repository access.

Observed errors included:

```
wget exited with error 1
unexpected end of file
```

Further investigation revealed that BusyBox wget lacked HTTPS support:

```
wget https://downloads.openwrt.org/
HTTPS support not compiled in.
```

This explained repeated failures retrieving APK repository metadata over HTTPS.

The issue represented a significant operational weakness because administrators were effectively forced to downgrade repository transport from HTTPS to HTTP in order to restore package functionality.

Potential security implications included:

- Man-in-the-middle attacks
- Package tampering
- Repository spoofing
- Supply-chain compromise

D. Temporary Package Recovery

HTTPS functionality was partially restored by manually transferring APK packages into the virtual machine and installing them locally:

```
apk add --allow-untrusted ./*.apk
```

Repository transport was later temporarily modified from HTTPS to HTTP to restore repository index access.

After modification:

```
apk update
```

successfully completed.

Core administrative packages were then installed successfully:

```
apk add nano
apk add vim
apk add curl
apk add tcpdump
apk add htop
apk add bash
apk add luci
```

VI. LUCI WEB INTERFACE DEPLOYMENT

After installation of LuCI, the OpenWrt web interface was initially inaccessible through:

```
http://192.168.1.1
```

Diagnostic analysis using:

```
netstat -tlnp
```

confirmed that the web server was actively listening on ports 80 and 443:

```
0.0.0.0:80 LISTEN
0.0.0.0:443 LISTEN
```

Further analysis revealed that the Windows host system could only directly access the VMware NAT interface rather than the internal OpenWrt LAN bridge.

The LuCI interface became accessible using:

```
http://192.168.111.130
```

rather than the internal bridge address.

This demonstrated the distinction between:

- Internal OpenWrt LAN addressing
- Externally reachable VMware NAT addressing
- Hypervisor network segmentation behavior

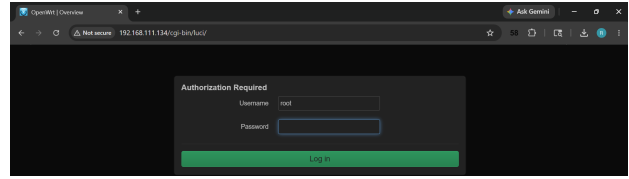


Fig. 2. Accessible LuCI web management login page

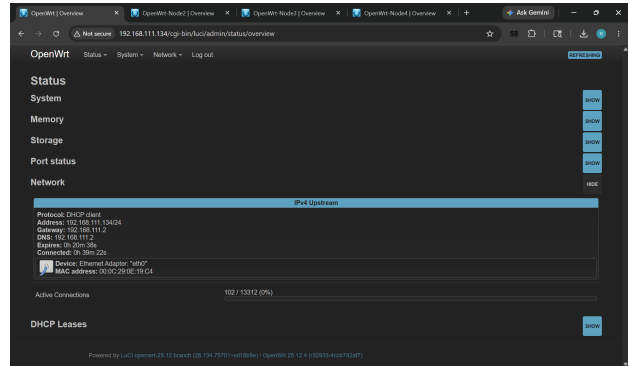


Fig. 3. Accessible LuCI web management interface through VMware NAT networking.

VII. MULTI-NODE MESH BACKBONE CONSTRUCTION

A. Dual-Interface Mesh Backbone

After establishing a stable baseline, a second virtual adapter was reintroduced using VMware host-only networking.

The second interface was assigned as a dedicated mesh backbone network:

```
uci set network.mesh=interface
uci set network.mesh.proto='static'
uci set network.mesh.device='eth1'
uci set network.mesh.ipaddr='10.10.10.1'
uci set network.mesh.netmask='255.255.255.0'
uci commit network
/etc/init.d/network restart
```

Resulting topology:

TABLE II
DUAL-INTERFACE OPENWRT TOPOLOGY

Interface	Address	Purpose
eth0	192.168.111.x	VMware NAT / WAN
eth1	10.10.10.x	Internal mesh backbone

Routing verification confirmed correct segmentation:

```
default via 192.168.111.2 dev eth0
10.10.10.0/24 dev eth1
```

```
root@OpenWrt:~# ip route
default via 192.168.111.2 dev eth0 src 192.168.111.134
10.10.10.0/24 dev eth1 scope link src 10.10.10.1
192.168.1.0/24 dev eth0 scope link src 192.168.1.1
192.168.111.0/24 dev eth0 scope link src 192.168.111.134
root@OpenWrt:~#
```

Fig. 4. Validated OpenWrt Node 1 routing table showing WAN and mesh backbone separation.

B. VM Cloning Strategy

Instead of reinstalling OpenWrt repeatedly, a stable “golden image” virtual machine was cloned using VMware full-clone functionality.

Resulting nodes included:

- OpenWrt-Node1
- OpenWrt-Node2
- OpenWrt-Node3
- OpenWrt-Node4

Each node received:

- Unique hostname
- Unique mesh backbone address
- VMware-generated MAC address

C. Four-Node Mesh Backbone Validation

The resulting mesh backbone addressing scheme included:

TABLE III
FOUR-NODE MESH BACKBONE ADDRESSING

Node	Mesh Address
OpenWrt-Node1	10.10.10.1
OpenWrt-Node2	10.10.10.2
OpenWrt-Node3	10.10.10.3
OpenWrt-Node4	10.10.10.4

Connectivity testing was performed between all nodes using ICMP echo requests.

Successful bidirectional communication was verified between all nodes with:

- Zero packet loss
- Stable latency
- Successful route propagation

The successful validation demonstrated:

- Stable Layer 2 adjacency
- Functional Layer 3 routing
- Operational VMware host-only backbone networking
- Successful multi-node OpenWrt virtualization

```
root@OpenWrt-Node4:~# ping -c 2 10.10.10.1
PING 10.10.10.1 (10.10.10.1) 56(84) bytes of data.
64 bytes from 10.10.10.1: icmp_seq=1 ttl=64 time=2.18 ms
64 bytes from 10.10.10.1: icmp_seq=2 ttl=64 time=1.33 ms

--- 10.10.10.1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1004ms
rtt min/avg/max/mdev = 1.334/1.755/2.176/0.421 ms
root@OpenWrt-Node4:~# ping -c 2 10.10.10.2
PING 10.10.10.2 (10.10.10.2) 56(84) bytes of data.
64 bytes from 10.10.10.2: icmp_seq=1 ttl=64 time=2.23 ms
64 bytes from 10.10.10.2: icmp_seq=2 ttl=64 time=4.85 ms

--- 10.10.10.2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1003ms
rtt min/avg/max/mdev = 2.234/3.542/4.851/1.308 ms
root@OpenWrt-Node4:~# ping -c 2 10.10.10.3
PING 10.10.10.3 (10.10.10.3) 56(84) bytes of data.
64 bytes from 10.10.10.3: icmp_seq=1 ttl=64 time=1.92 ms
64 bytes from 10.10.10.3: icmp_seq=2 ttl=64 time=1.48 ms

--- 10.10.10.3 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1003ms
rtt min/avg/max/mdev = 1.480/1.700/1.921/0.220 ms
root@OpenWrt-Node4:~#
```

Fig. 5. Successful four-node mesh backbone connectivity validation.

D. Sequential Startup and DHCP Convergence

Initial simultaneous startup of cloned nodes produced temporary WAN IP duplication behavior within the VMware NAT network.

Multiple nodes briefly inherited identical DHCP leases from previous cloned states.

This behavior was mitigated by sequential node startup. Sequential startup allowed VMware DHCP services to converge properly and assign unique WAN addresses based on MAC address differentiation.

Final WAN assignments stabilized as:

- Node1: 192.168.111.134
- Node2: 192.168.111.136
- Node3: 192.168.111.137
- Node4: 192.168.111.138

This demonstrated:

- DHCP lease convergence behavior
- Temporary ARP instability during cloned startup
- Virtualization timing dependencies
- Successful MAC-based lease differentiation

E. IPv6 Duplicate Address Detection Behavior

During sequential startup of cloned nodes, OpenWrt repeatedly reported IPv6 duplicate address detection warnings:

IPv6 duplicate address detected

Despite repeated IPv6 duplicate detection events:

- IPv4 routing remained stable
- Mesh backbone communication remained operational
- Node-to-node connectivity was unaffected

This behavior represents an important operational consideration for cloned embedded router deployments and virtualized distributed networking systems.

```
[ 12.533917] procd: - init -
Please press Enter to activate this console.
[ 17.759256] kmodloader: loading kernel modules from /etc/modules.d/*
[ 17.790095] urngd: v1.0.2 started.
[ 17.851278] e1000: Intel(R) PRO/1000 Network Driver
[ 17.859459] e1000: Copyright (c) 1999-2006 Intel Corporation.
[ 18.500079] e1000 0000:02:00:0 eth0: (PCI:66MHz:32-bit) 00:0c:29:8a:68:7e
[ 18.500535] e1000 0000:02:00:0 eth0: Intel(R) PRO/1000 Network Connection
[ 19.141860] e1000 0000:02:01:0 eth1: (PCI:66MHz:32-bit) 00:0c:29:8a:68:80
[ 19.147917] e1000 0000:02:01:0 eth1: Intel(R) PRO/1000 Network Connection
[ 19.165791] ixgbe: Intel(R) 10 Gigabit PCI Express Network Driver
[ 19.172232] ixgbe: Copyright (c) 1999-2016 Intel Corporation.
[ 19.105750] i2c_dev: i2c /dev entries driver
[ 19.237308] PPP generic driver version 2.4.2
[ 19.244786] NET: Registered PF_PPPOX protocol family
[ 19.364798] kmodloader: done loading kernel modules from /etc/modules.d/*
[ 21.753403] 8021q: adding VLAN 0 to HW filter on device eth0
[ 21.764898] e1000: eth0 NIC Link is Up 1000 Mbps Full Duplex, Flow Control: N
one
[ 22.005832] 8021q: adding VLAN 0 to HW filter on device eth1
[ 22.013822] e1000: eth1 NIC Link is Up 1000 Mbps Full Duplex, Flow Control: N
one
[ 22.886777] IPv6: eth0: IPv6 duplicate address fdd8:a689:d40c:1 used by 00:0c:29:8a:19:c4 detected!
```

Fig. 6. Observed IPv6 duplicate address detection during sequential startup of cloned OpenWrt nodes.

VIII. BATMAN-ADV MESH ROUTING DEPLOYMENT

A. BATMAN-adv Installation

After the successful construction of the four-node OpenWrt backbone topology, the BATMAN-adv routing protocol was deployed to provide dynamic Layer 2 mesh routing functionality.

The following packages were installed on all nodes:

```
apk add kmod-batman-adv batctl luci-proto-batman-adv
```

Successful installation loaded the BATMAN kernel module dynamically:

```
lsmod | grep batman
```

The BATMAN mesh interface was manually initialized:

```
modprobe batman-adv
ip link add name bat0 type batadv
ip link set up dev bat0
```

The dedicated mesh transport interface was attached to BATMAN:

```
batctl meshif bat0 if add eth1
```

```
root@OpenWrt:~# modprobe batman-adv
root@OpenWrt:~# ip link add name bat0 type batadv
root@OpenWrt:~# ip link set up dev bat0
[ 4339.498563] 8021q: adding VLAN 0 to HW filter on device bat0
root@OpenWrt:~# ip link show bat0
4: bat0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UNKNOWN
qlen 1000
    link/ether 8e:7f:35:59:1c:ce brd ff:ff:ff:ff:ff:ff
root@OpenWrt:~#
```

Fig. 7. Successful initialization of the BATMAN-adv virtual mesh interface.

B. Dynamic Neighbor Discovery

BATMAN dynamically discovered neighboring routers using:

```
batctl meshif bat0 n
```

Originator propagation tables were observed using:

```
batctl meshif bat0 o
```

```
root@OpenWrt-Node3:~# batctl meshif bat0 n
[B.A.T.M.A.N. adv 2025.4-openwrt-4, MainIF/MAC: eth1/00:0c:29:8a:68:80 (bat0/Se:
da:e2:a5:06:84 BATMAN_IV)]
IF Neighbor last-seen
eth1 00:0c:29:a7:71:c8 0.040s
eth1 00:0c:29:0e:19:ce 0.200s
root@OpenWrt-Node3:~# batctl meshif bat0 o
[B.A.T.M.A.N. adv 2025.4-openwrt-4, MainIF/MAC: eth1/00:0c:29:8a:68:80 (bat0/Se:
da:e2:a5:06:84 BATMAN_IV)]
Originator last-seen (#/255) Nexthop [outgoingIF]
00:0c:29:a7:71:c8 0.000s ( 82) 00:0c:29:0e:19:ce [ eth1 ]
* 00:0c:29:a7:71:c8 0.000s ( 84) 00:0c:29:a7:71:c8 [ eth1 ]
00:0c:29:0e:19:ce 0.020s ( 82) 00:0c:29:a7:71:c8 [ eth1 ]
* 00:0c:29:0e:19:ce 0.020s ( 93) 00:0c:29:0e:19:ce [ eth1 ]
root@OpenWrt-Node3:~#
```

Fig. 8. BATMAN-adv originator routing tables dynamically populated across the mesh.

C. Four-Node Mesh Convergence

All four OpenWrt nodes successfully joined the distributed BATMAN mesh.

The resulting deployment demonstrated:

- Autonomous neighbor discovery
- Dynamic route propagation
- Persistent distributed convergence
- Self-healing mesh behavior

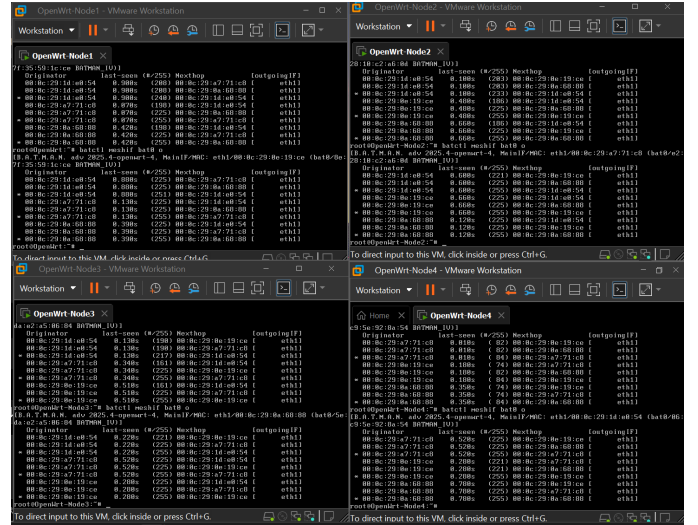


Fig. 9. Stable four-node BATMAN-adv mesh topology before failure testing.

D. Node Failure and Reconvergence

One node was intentionally powered off during operation to evaluate mesh resilience behavior.

Immediately after failure:

- Neighbor tables updated dynamically
- Stale originators were removed
- Remaining nodes maintained connectivity

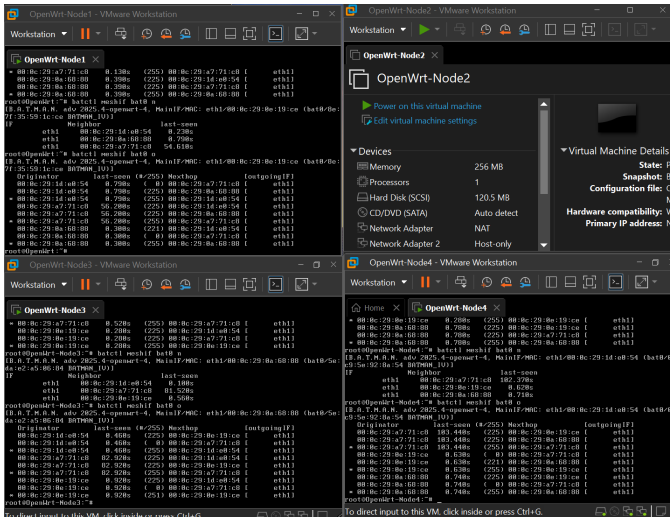


Fig. 10. BATMAN topology immediately following intentional node failure.

E. Node Reintegration

After reboot, the failed node automatically rejoined the distributed mesh without manual reconfiguration.

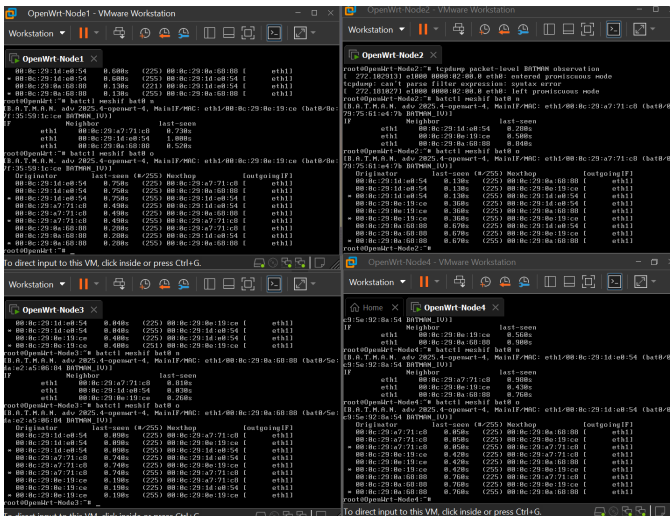


Fig. 11. Automatic BATMAN mesh reconvergence after node restoration.

F. Persistent BATMAN Configuration

Persistent deployment was implemented using OpenWrt UCI configuration:

```
uci set network.bat0=interface
uci set network.bat0.proto='batadv'
uci set network.bat0.routing_algo='BATMAN_IV'
```

```
uci set network.mesh_bat=interface
uci set network.mesh_bat.proto='batadv_hardi
uci set network.mesh_bat.master='bat0'
uci set network.mesh_bat.device='eth1'
```

```
uci commit network
```

After reboot, all nodes automatically:

- Recreated bat0
- Reattached eth1
- Rediscovered neighbors
- Rejoined the mesh

G. BATMAN Native Ping and Traceroute

BATMAN-native connectivity was validated using:

```
batctl meshif bat0 ping <neighbor MAC>
```

Traceroute functionality was validated using:

```
batctl meshif bat0 traceroute <neighbor MAC>
```

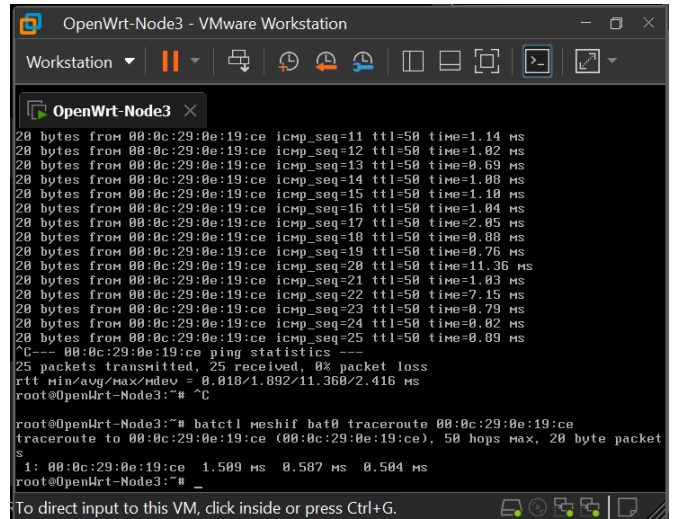


Fig. 12. BATMAN-native traceroute across the distributed mesh topology.

H. MTU Optimization and Fragmentation Analysis

BATMAN generated warnings indicating insufficient MTU allocation on the transport interface.

Initial configuration:

- eth1 MTU: 1500
- bat0 MTU: 1500

The transport interface MTU was increased experimentally:

```
ip link set dev eth1 mtu 1532
```

Following optimization:

- Neighbor discovery remained stable
- BATMAN forwarding persisted
- Packet loss remained zero

Interestingly, bat0 continued operating at MTU 1500 while eth1 operated at MTU 1532.

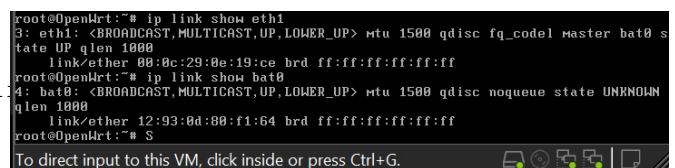


Fig. 13. Initial BATMAN deployment with insufficient MTU allocation.


```
sed -i 's/\r$//' /root/export-config.sh
```

This represented an important interoperability consideration between Windows administrative systems and embedded Linux networking environments.

C. SSH and SCP Artifact Extraction

Configuration archives were remotely extracted from OpenWrt nodes using SCP over SSH.

Modern Windows SCP implementations attempted to default to SFTP mode, which was unsupported by the lightweight OpenWrt Dropbear deployment.

Observed error:

```
subsystem request failed on channel 0
scp: Connection closed
```

Successful extraction required forcing legacy SCP protocol mode:

```
scp -O root@192.168.111.x:/root/archive.tar.gz
```

This demonstrated another operational limitation common within lightweight embedded Linux distributions where full SFTP subsystems are often omitted to conserve storage space and memory resources.

D. Future Inter-Group Mesh Expansion

Future work may interconnect independently deployed BATMAN mesh clusters between project groups using encrypted WireGuard tunnels.

Potential architecture:

- Independent BATMAN mesh clusters
- WireGuard encrypted overlay tunnels
- Distributed inter-group route propagation
- Geographically separated mesh federation

This architecture would allow BATMAN-adv to operate across encrypted Internet-based transport overlays while preserving autonomous distributed mesh behavior.

X. FUTURE RASPBERRY PI DEPLOYMENT

The final implementation stage will migrate the virtualized mesh environment onto Raspberry Pi hardware devices running OpenWrt and LibreMesh.

The Raspberry Pi deployment will evaluate:

- Physical wireless mesh formation
- Real-world throughput and latency
- Hardware limitations
- Deployment obstacles
- Inter-group node interoperability

XI. UNRESOLVED OPERATIONAL CHALLENGES AND FUTURE INVESTIGATION

Although the distributed BATMAN-adv mesh deployment operated successfully overall, several operational inconsistencies and unresolved behaviors were identified during experimentation.

These issues represent valuable future investigation areas for both virtualized and physical mesh deployments.

A. Persistent VMware DHCP Address Duplication

Despite successful BATMAN mesh convergence, VMware NAT DHCP services occasionally assigned identical WAN-side addresses to multiple cloned OpenWrt nodes.

Example observed behavior included:

- Multiple nodes simultaneously assigned 192.168.111.141
- Ambiguous SSH access
- Non-deterministic LuCI management behavior
- Administrative-plane collisions

Although sequential startup reduced the frequency of collisions, duplicate assignments still occasionally reappeared during extended experimentation.

The issue appeared isolated to VMware NAT-side DHCP behavior and did not affect BATMAN forwarding functionality operating on the dedicated mesh backbone interface.

This behavior represents an important virtualization-management limitation requiring further investigation.

B. IPv6 Duplicate Address Detection Events

OpenWrt repeatedly reported IPv6 duplicate address detection warnings during sequential startup and node cloning operations:

```
IPv6 duplicate address detected
```

Despite repeated warnings:

- IPv4 routing remained operational
- BATMAN mesh convergence persisted
- Node-to-node communication remained stable

The project did not fully investigate:

- IPv6 address autoconfiguration behavior
- Duplicate IPv6 neighbor advertisements
- IPv6 interactions with BATMAN-adv
- Potential virtualization-layer MAC/address reuse

Future work should evaluate whether disabling IPv6 entirely or implementing static IPv6 addressing would improve deployment stability.

C. Management-Plane and Forwarding-Plane Separation

The project revealed significant architectural separation between:

- Administrative management-plane behavior
- BATMAN distributed forwarding behavior

Even during severe management-plane inconsistencies:

- BATMAN neighbor discovery remained stable
- Originator propagation continued
- Distributed forwarding functionality persisted

This highlighted the resilience of the distributed forwarding plane while simultaneously exposing administrative fragility caused by virtualization and DHCP interactions.

D. Lightweight Embedded Linux Limitations

Several OpenWrt operational limitations were identified during experimentation:

- Missing HTTPS support in BusyBox wget
- Missing SFTP subsystem support in Dropbear
- Minimal shell-tool availability
- CRLF/LF shell-script compatibility issues

These limitations complicated:

- Package installation
- Secure artifact transfer
- Deployment automation
- Remote administration

While expected for lightweight embedded environments, these limitations introduced significant operational overhead during deployment and experimentation.

E. Incomplete Multi-Hop Routing Validation

Although the BATMAN mesh operated successfully across all four nodes, the VMware host-only topology allowed all routers to remain directly adjacent at Layer 2.

As a result, the deployment did not fully validate:

- True multi-hop relay behavior
- Intermediate-node forwarding dependency
- Mesh partition recovery
- Indirect route optimization

Future work should intentionally isolate subsets of nodes to force multi-hop forwarding paths and evaluate true relay-based mesh behavior.

F. Future WireGuard-Based Inter-Mesh Federation

Future experimentation may connect independent BATMAN mesh deployments between project groups using encrypted WireGuard overlay tunnels.

Potential future architecture:

- Independent local BATMAN clusters
- WireGuard encrypted Internet overlays
- Distributed inter-group route propagation
- Geographically separated mesh federation

This would allow evaluation of:

- Distributed mesh scalability
- Internet-based overlay transport
- Secure inter-cluster communication
- Hybrid mesh-overlay architectures

XII. CONCLUSION

This project successfully implemented a persistent four-node distributed OpenWrt mesh-routing laboratory environment using VMware virtualization and BATMAN-adv.

The completed deployment demonstrated:

- Successful OpenWrt virtualization
- Stable multi-node routed mesh backbone construction
- Autonomous BATMAN neighbor discovery
- Dynamic distributed originator propagation
- Persistent mesh deployment through UCI configuration

- Autonomous reconvergence after node failure
- BATMAN-native ping and traceroute functionality
- MTU optimization and fragmentation analysis
- Packet-level mesh observation using tcpdump
- Identification of virtualization and management-plane edge cases

The project evolved significantly beyond initial router virtualization objectives and ultimately produced a functional distributed autonomous mesh-routing research platform suitable for continued LibreMesh experimentation and future Raspberry Pi deployment.

The resulting environment now supports:

- Reproducible distributed networking experimentation
- Dynamic routing analysis
- Mesh-failure resilience evaluation
- Multi-node convergence observation
- Future wireless mesh deployment research

The completed environment represents a robust foundation for continued experimentation with decentralized community-networking infrastructure using LibreMesh and BATMAN-adv.

REFERENCES

- [1] OpenWrt Project, "OpenWrt Documentation," [Online]. Available: <https://openwrt.org>
- [2] LibreMesh Project, "LibreMesh Documentation," [Online]. Available: <https://libremesh.org>
- [3] BMX7 Routing Protocol, "BMX7 GitHub Repository," [Online]. Available: <https://github.com/bmx7-routing/bmx7>
- [4] BATMAN-adv Project, "B.A.T.M.A.N. advanced Documentation," [Online]. Available: <https://www.open-mesh.org/projects/batman-adv/wiki>
- [5] Open-Mesh, "batctl User Guide," [Online]. Available: <https://www.open-mesh.org/projects/batman-adv/wiki/Batctl>
- [6] VMware Inc., "VMware Workstation Pro Documentation," [Online]. Available: <https://docs.vmware.com>

APPENDIX A

EXPORTED CONFIGURATION AND OPERATIONAL ARTIFACTS

The following exported artifacts were generated and archived from each OpenWrt node:

- network-nodeX
- system-nodeX
- firewall-nodeX
- dhcp-nodeX
- batman-if-nodeX.txt
- batman-neighbors-nodeX.txt
- batman-origimators-nodeX.txt
- ip-addr-nodeX.txt
- ip-route-nodeX.txt
- packages-nodeX.txt

These artifacts provide reproducible deployment state information and operational validation for the completed distributed BATMAN-adv mesh environment.